5. Dimmen einer LED mittels Pulsweitenmodulation (PWM)

Bisher haben wir LEDs ein- und wieder ausgeschaltet. Für viele Anwendungen ist es wünschenswert, neben den beiden Zuständen „Ein“ und „Aus“, auch Zwischenstufen (Helligkeiten) realisieren zu können. Dazu benötigen wir eine Möglichkeit, die Ausgangsspannung zu regulieren. Da der Arduino nicht über einen eingebauten Digital-Analog Wandler verfügt, bietet sich die sogenannte „Pulsweitenmodulation“ an. Ein normaler digitaler Ausgang wird dabei periodisch sehr schnell ein- und ausgeschaltet. Die LED wird umso heller leuchten, je länger der Ausgang einen „High“-Zustand annimmt (im Verhältnis zu der Zeit im „Low“-Zustand). Abbildung 5.2 veranschaulicht dieses Prinzip.

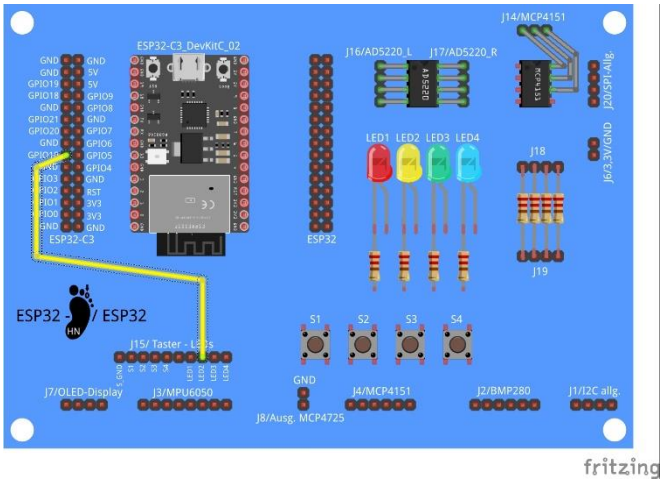


Abbildung 5.1: LED mit Pulsweitenmodulation über Poti gesteuert

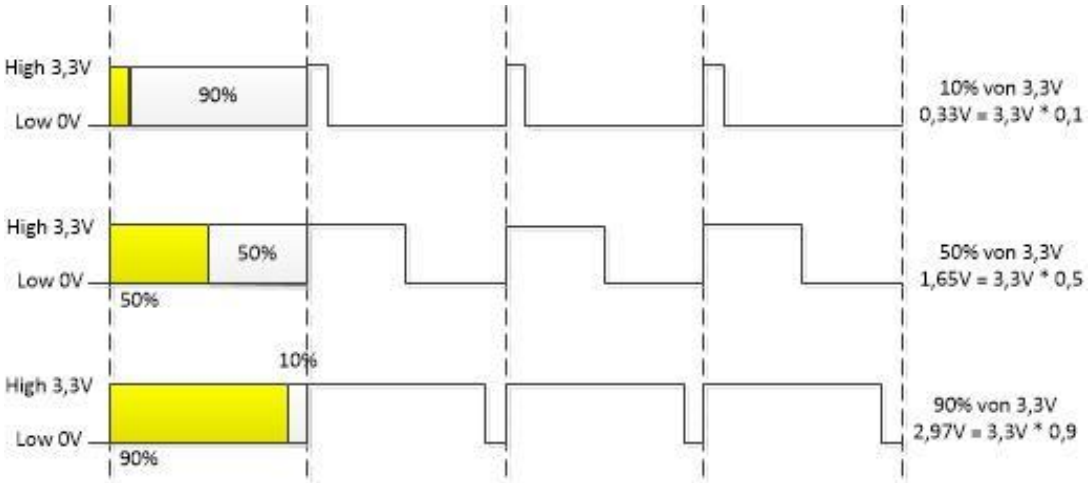


Abbildung 5.2: Pulsweiten-Modulation

GT©

Nutzen Sie folgenden Code zum Test der PWM-Funktionalität:

```
/* PWM: Die Helligkeit einer Leuchtdiode wird über PWM gesteuert. */

int gruen = 10;                         // Port der gruenen LED definieren
int helligkeit = 128;                   // Helligkeitswert definieren

void setup()
{ }

void loop()
{
        analogWrite(gruen, helligkeit);         // Port 10 gesetzt.
}
```

Die LED sollte jetzt weniger hell leuchten als zuvor.

Aufgaben

- Testen Sie die Leuchtstärke für verschiedene Werte der Variable *helligkeit*. Die Funktion *analogWrite* akzeptiert Werte im Bereich 0 bis 255.

- Programmieren Sie einen „Sweep“-Modus. Die Helligkeit soll langsam zunehmen, bis sie den Maximalwert erreicht. Sie können dazu ihr Progamm um folgendes Codefragment ergänzen:

```
for (int helligkeit=0; helligkeit <= 255; helligkeit++)
{
        analogWrite(gruen, helligkeit);
        delay(25);                      // Warte 25 ms.
}
```

- (optional:) Erweitern Sie Ihr Programm so, dass die LED periodisch heller und dunkler wird.

6. Motor in Betrieb nehmen

Jetzt wollen wir nach demselben Prinzip anstelle der LED einen Motor steuern. Wir verwenden hier einen Gleichstrommotor, der auch mit niedrigen Spannungen arbeitet. Gleichstrommotoren werden vielfältig eingesetzt, beispielsweise bei Lüftern im Computer, als Antrieb bei Modellautos (Carrera Bahn), usw.

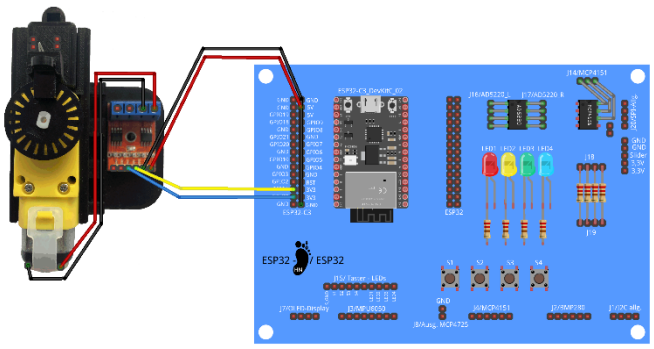


Abbildung 6.1:

Der Betrieb des Motors erfordert einen höheren Strom, als dies bei den LEDs der Fall gewesen ist. Die maximale Belastung eines Pin (GPIO) beträgt 40mA. Aus diesem Grunde benutzen wir ein Motor-Treiber Board zur Stromversorgung. Das Motor-Treiber Board L9110 Dual-DC kann zwei Motoren über jeweils 2 GPIO's ansteuern.

Die Ansteuerung der L9110 können Sie der nachfolgenden Tabelle entnehmen.

	1A1	1B1
Vorwärts volle Geschwindigkeit	Low	High
Rückwärts volle Geschwindigkeit	High	Low
Vorwärts gesteuerte Geschw.	Low	PWM (0…255)
Rückwärts gesteuerte Geschw.	PWM (0..255)	Low
Stopped	Low	Low
Stopped	High	High

Das nachfolgende Programm arbeitet noch ohne PWM, es schaltet den Motor in den Rechtslauf (volle Geschwindigkeit) für 5 sec. Dann wird der Motor gestoppt. Nach 2 sec. wird der Motor in den Linkslauf (volle Geschwindigkeit) geschaltet, ebenfalls für 5 sec. Dann stoppt der Motor wieder.

Programm: Motor Rechtslauf / Linkslauf

```
int B_1 = 0;     // 0 = GPIO0
int A_1 = 1;     // 1 = GPIO1

void setup()
{
   pinMode(B_1,OUTPUT);
   pinMode(A_1,OUTPUT);
```

```
}
void loop()
{
   digitalWrite(B_1,LOW);
   digitalWrite(A_1,HIGH);
   delay(5000);
   digitalWrite(B_1,LOW);
   digitalWrite(A_1,LOW);
   delay(2000);
   digitalWrite(B_1,HIGH);
   digitalWrite(A_1,LOW);
   delay(5000);
   digitalWrite(B_1,LOW);
   digitalWrite(A_1,LOW);
   delay(2000);
}
```

Bauen Sie die Schaltung aus Abbildung 6.1 auf.

Aufgabe:

- Nehmen Sie den Motor in Betrieb.
  Verwenden Sie das Programm *Motor Rechtslauf / Linkslauf*

7. Motorsteuerung

Nun soll der Motor mit unterschiedlichen Geschwindigkeiten gesteuert werden. Dazu verwenden wir nicht mehr das „digitalWrite“, welches nur die Werte LOW=0 und HIGH=255 an den Motor schicken kann. Zum Einsatz kommt nun der Befehl „analogWrite“, mit diesem Befehl lassen sich alle Werte zwischen 0 und 255 übertragen, so dass der Motor mit unterschiedlichen Geschwindigkeiten betrieben werden kann.

Ändern Sie hierzu das letzte Programm wie folgt ab:

```
int B_1 = 0;     // 0 = GPIO0
int A_1 = 1;     // 1 = GPIO1
int LS= 160;        lowest Speed
int MS= 200;        middle Speed
int HS= 255;        highest Speed
```

```
void setup()
{}

void loop()
{
  analogWrite(B_1,LS);
  analogWrite(A_1,0);
  delay(1000);
       .
       .
       .
  delay(1000);
}
```

Aufgaben:

- Erweitern Sie das Programm so, dass der Motor nacheinander in einer niedrigen- mittleren- und hohen Geschwindigkeit läuft.
- Erweitern Sie das Programm so, dass der Motor langsam hochläuft, d.h. es sollen die Werte von 0 bis 255 mit einer Zeitverzögerung von ca. 50ms übertragen werden. Recht einfach lässt sich dies mit einer for-Schleife umsetzen.
  for (i=0; i < 255; i++)
- Ergänzen Sie das Programm insoweit, dass der Motor nach dem langsamen Hochlauf, auch wieder langsam seine Geschwindigkeit bis auf null reduziert, d.h. es wird eine zweite for-Schleife benötigt.
  for (i=255; i > 0; i--)

Motor-Steuerung per Smartphone

Scannen Sie mit Ihrem Smartphone den für Sie passenden QR-Code.

Installieren Sie die App “RemoteXY”







Erste App generieren

Über die Webseite https://remotexy.com lassen sich Steuerungs-Oberflächen für das Smartphone erstellen, deren Code wir in unseren Programmen verwenden können. Für unsere Anwendung wurde ein Slider und ein Switch verwendet. Der daraus resultierende Code ist nachfolgend dargestellt.

Einige Erläuterungen zum Beispiel-Code.

```
/*
   -- Motor 1 --

   This source code of graphical user interface
   has been generated automatically by RemoteXY editor.
   To compile this code using RemoteXY library 3.1.13 or later version
   download by link http://remotexy.com/en/library/
       To connect using RemoteXY mobile app by link
       http://remotexy.com/en/download/
     - for ANDROID 4.14.08 or later version;
     - for iOS 1.11.2 or later version;
```

```
   This source code is free software; you can redistribute it and/or
   modify it under the terms of the GNU Lesser General Public
   License as published by the Free Software Foundation; either
   version 2.1 of the License, or (at your option) any later version.
*/

//////////////////////////////////////////////
//        RemoteXY include library          //
//////////////////////////////////////////////

// you can enable debug logging to Serial at 115200
//#define REMOTEXY__DEBUGLOG

// RemoteXY select connection mode and include library
#define REMOTEXY_MODE__ESP32CORE_BLE

#include <BLEDevice.h>

// RemoteXY connection settings
#define REMOTEXY_BLUETOOTH_NAME "Motor"
#define REMOTEXY_ACCESS_PASSWORD "test"


#include <RemoteXY.h>

// RemoteXY GUI configuration
#pragma pack(push, 1)
uint8_t RemoteXY_CONF[] =   // 44 bytes
  {
      255,2,0,0,0,37,0,18,0,0,0,31,1,106,200,1,1,2,0,4,
      45,1,20,159,48,2,26,2,38,174,44,22,0,2,26,31,31,79,78,0,
      79,70,70,0
  };

// this structure defines all the variables and events of your control interface
struct {

    // input variables
  int8_t slider_01; // from -100 to 100
  uint8_t switch_01; // =1 if switch ON and =0 if OFF

    // other variable
  uint8_t connect_flag;  // =1 if wire connected, else =0

} RemoteXY;
#pragma pack(pop)

//////////////////////////////////////////////
//           END RemoteXY include           //
//////////////////////////////////////////////

void setup()
{
  RemoteXY_Init ();
  // TODO you setup code

}

void loop()
```

Name der Bluetooth-Verbindung auf dem Smartphone

Hier können Sie das Passwort ändern

Diesen Bereich bitte NICHT verändern, da sonst die APP nicht mehr funktioniert.

Hier finden Sie alle definierten Variablen die auf dem Smartphone angezeigt werden

Mit dieser Variablen kann überprüft werden ob eine Bluetooth-Verbindung besteht.

Name der Daten-Struktur, diese wird benötigt um über die „Punkt-Notation“ auf die Variablen zugreifen zu können

Hier können Sie die Befehle platzieren, die im „void setup“ ausgeführt werden müssen.

```
{
  RemoteXY_Handler ();
  // TODO you loop code
  // use the RemoteXY structure for data transfer
  // do not call delay(), use instead RemoteXY_delay()


}
```

Platz für Ihr Hauptprogramm

Beispiel für den Zugriff auf die struct-Variablen

Die Daten des Sliders können Sie mit dem folgenden Befehl auslesen
int Motor = RemoteXY.slider_01;

„RemoteXY“ ist der Name der Struktur in der sich die Variable „slider_01“ befindet.

Erster Test

Laden Sie über moodle / Erstsemester-Projekt den Sketch „RemoteXY-Slider.ino“ herunter, und öffnen Sie diesen Sketch in der Arduino-IDE.

Ändern Sie bitte die folgenden Variablen:

REMOTEXY_BLUETOOTH_NAME = Wählen Sie einen anderen Namen, NICHT Motor

REMOTEXY_ACCESS_PASSWORD = Vergeben Sie hier ein neues Passwort

Wenn die Einstellungen in Ihrer Arduino-IDE bzgl. Board (ESP32C3 Dev Module) und Port korrekt sind, können Sie das Programm nun zum Controller uploaden.

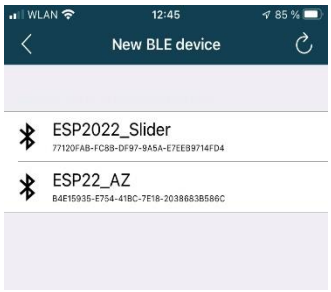
Nach erfolgtem Upload, öffnen Sie bitte Ihr Smartphone und starten die App „RemoteXY“. Die App sollte sich mit dem folgenden Screen melden. Da beim ersten Start noch keine Verbindung besteht, erzeugen Sie ein „neues Device“ durch Anklicken des +



In der folgenden Abbildung wählen Sie bitte „Bluetooth LE“.



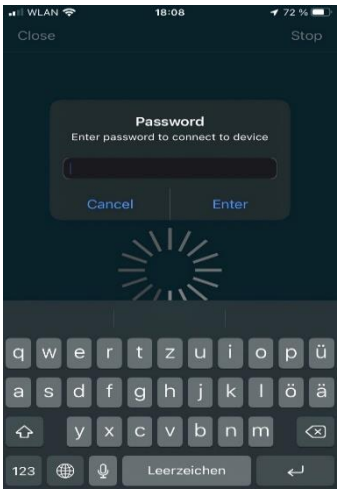
Wenn sich andere Gruppen in der Nähe befinden werden Ihnen nicht nur Ihre Verbindung (ESP2022_Slider) angeboten. Wählen Sie bitte Ihre Bluetooth-Verbindung, durch Anklicken, aus. Den Bluetooth-Verbindungsnamen können Sie im Quellcode in der Variablen „REMOTEXY_BLUETOOTH_NAME“ ändern. Wählen Sie einen Namen unter dem sie Ihr Fahrzeug eindeutig identifizieren können.



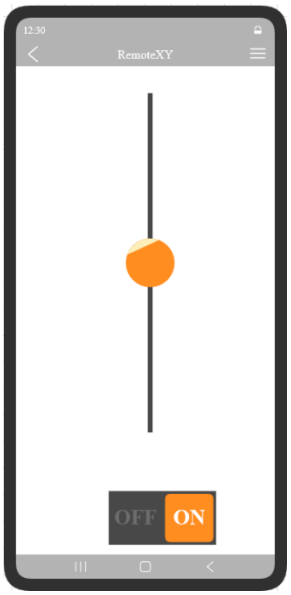
Besteht bereits eine Verbindung erhalten Sie die folgende Darstellung auf Ihrem Smartphone und können durch Anklicken die Verbindung herstellen.

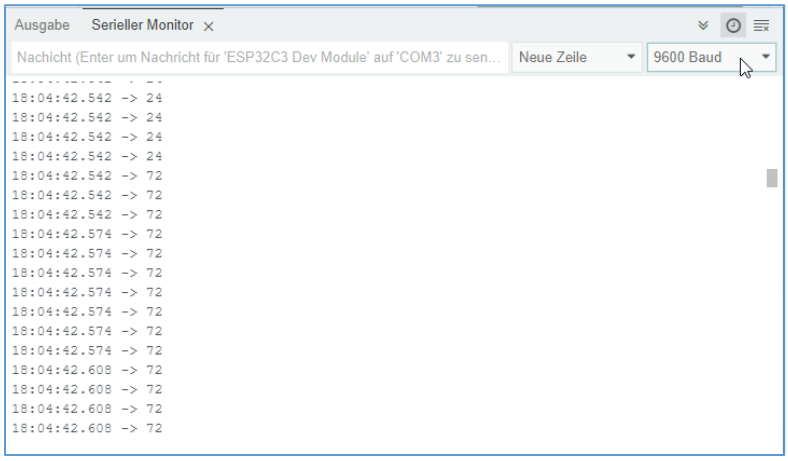


Wenn Sie im Quellcode, in der Variablen „REMOTEXY_ACCESS_PASSWORD“ ein Passwort vergeben haben, werden Sie jetzt dazu aufgefordert sich zu legitimieren.
Die Verwendung eines Passwortes ist ratsam, da sich möglicherweise andere Gruppen mit Fahrzeugen in der Nähe befinden und bei einer evtl. Namensgleichheit Ihre Verbindung von einer anderen Gruppe übernommen werden könnte.



Die Beispiel-Applikation müsste sich wie nebenstehend melden. Wenn Sie nun den Slider bewegen, werden Werte zwischen -100 und 100 an den Mikrocontroller gesendet, je nachdem ob Sie den Slider nach oben oder unten bewegen.





Aufgaben:

- Schalten Sie den ON/OFF Switch ein.
- Betätigen Sie den Slider auf Ihrem Smartphone. Der Motor sollte nun je nach Stellung des Sliders Vorwärts- oder Rückwärts laufen, mit unterschiedlichen Geschwindigkeiten.
- Öffnen Sie den seriellen Monitor in der Arduino-IDE. Betätigen Sie den Slider auf Ihrem Smartphone. Sie sollten nun die Wertänderungen, die der Slider generiert, beobachten können.
- Schalten Sie bei betätigtem Slider den Switch in die Position OFF. In der Anzeige sollte nun nur noch der Wert 0 erscheinen, und der Motor sollte still stehen.

Pinout des ESP32-C3_DevKitC_02 Boards

